0. Orientation: Programs, Computers, and Go

00. Orientation: Programs, Computers, and Go

Phase: Phase 0 — Orientation
Estimated time: 1 day
Slug: `00-orientation`

Learning objectives

- Explain what a program is and how source code becomes running software
- Describe CPU, memory, and the OS at a beginner level
- Contrast Go with Python (compiled, static types, concurrency model)
- Install the Go toolchain and run your first program
- Adopt the study method: attempt → struggle → solution → rewrite from memory

Prerequisites

Complete prior numbered lessons. Type every example yourself.

What is a program?

A **program** is a precise sequence of instructions a computer can execute. You write **source code** (human-readable text). A **compiler** or **interpreter** turns that into machine instructions the CPU understands.

- **CPU**: executes instructions one at a time (very fast).
- **Memory (RAM)**: temporary storage for values your program uses while running.
- **Disk**: durable storage (files, databases).
- **OS**: manages processes, files, networking; your Go program runs as a process.

Compiled vs interpreted (Go vs Python)

	Python	Go
How it runs	Interpreter / VM	Compiled to native machine code
Types	Dynamic (checked at runtime)	Static (checked at compile time)
Speed	Often slower for CPU work	Typically fast
Concurrency	GIL limits CPU threads	Goroutines + scheduler

Go catches many mistakes **before** your program runs. That feels stricter than Python at first — and that strictness is a feature for infrastructure work.

The Go toolchain

After installing Go (https://go.dev/dl/):

```
go version          # confirm install
go env GOPATH GOROOT
mkdir -p hello && cd hello
go mod init example.com/hello
```

Create `main.go`:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, Goodway")
}
```

```
go run .            # compile + run
go build -o hello   # produce a binary
./hello
go test ./...       # run tests (later lessons)
```

How to study this course

1. Read the lesson PDF carefully; type examples yourself (do not copy-paste blindly).
2. Attempt every exercise **without** opening `solutions/`.
3. If stuck >20 minutes, re-read the concept section, then try a smaller version.
4. Only then compare with the solution — write notes on *why* it works.
5. Next day: rewrite the solution from memory without looking.

Track progress in `curriculum/PROGRESS.md`.

Common mistakes (especially from Python instincts)

- Expecting Go to feel like Python (dynamic typing, exceptions everywhere)
- Skipping the rewrite-from-memory step — that is where mastery forms
- Installing multiple Go versions without understanding modules

Exercises

Work in `exercises/00-orientation/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/00-orientation/` until you have an honest failing attempt.

Self-check

1. What does the Go compiler do that a Python interpreter does differently?

2. Name three commands you will use daily (`go run`, `go test`, …).
3. Why does this course ban looking at solutions first?

Further reading

- https://go.dev/doc/install
- https://go.dev/tour/